

COMP726 Blockchain and Cryptocurrency Technology

Semester 2 2026

Individual Technical Assessment Portfolio

Due Date: 4th Oct at 23:59 pm

Exercises	1	2	3	4	5	6	Total
Marks	25	23	12	20	10	10	100

General Instructions:

1. Submit a single PDF document containing all answers.
2. Show all calculations and workings where applicable.
3. Screenshots must be clearly labeled, of sufficient resolution, and easy to read.
4. Code may be embedded directly in the document or provided through a GitHub repository link.
5. If using GitHub, ensure the repository is publicly accessible or access has been granted to the lecturers.
6. Any external sources, tools, or references used should be acknowledged.
7. Students should retain access to their wallet, deployed contract, and source code until final grades have been released.
8. Students may use AI tools to support learning; however, marks will be awarded for evidence of the student's own practical work, analysis, interpretation, and reflection.

Exercise 1 [25 marks]: Blockchain Structure and Tamper Detection

Create a simple blockchain in Python that satisfies the following requirements:

1. Design and create a “Block” class, based on the implementation demonstrated in class. You may modify or extend the class as needed.
2. Using SHA-256 hash algorithm to generate the hash for each block.
3. Design and create a “Blockchain” class. Your “Blockchain” class should include, but is not limited to, the following functionality:
 - Automatically creates a genesis block when the blockchain is initialised.
 - Allows new blocks to be appended.
 - Validates the integrity of the entire blockchain.
 - Reports the first block that fails validation.
4. Create a blockchain containing **10 blocks**, including the genesis block. Each block should contain different transaction data.
5. Test your code:
 - After the blockchain has been created, modify the transaction data stored in the 5th block without recalculating its hash.
 - Validate this blockchain and report the validation result.

Your answer must include:

- Screenshots of your complete Python source code. **[10 marks]**
- Screenshots showing the output of your code, including blockchain before the modification in Task 5. **[2 marks]**
- Screenshots showing the modified blockchain and the validation results. **[3 marks]**

Reflection Questions:

Answer the following questions based on your implementation, experimental results, and understanding of the underlying concepts. Support your answers with evidence where appropriate.

1. Explain why modifying the transaction data in the 5th block causes the subsequent blocks to become invalid. **[2 marks]**
2. What information should a blockchain validation program report when blockchain corruption or tampering is detected? Explain why each piece of information is useful. **[3 marks]**
3. Suggest one improvement that would make your blockchain more resistant to tampering. Explain how your proposed improvement enhances the security or integrity of the blockchain. **[5 marks]**

Exercise 2 [23 marks]: Proof of Work: Mining, Difficulty & Probability

Implement Proof of Work using SHA-256. The mining algorithm should continue incrementing the nonce until the block hash begins with a specified number of leading zeros. Test the following difficulty levels:

- 2 leading zeros
- 3 leading zeros
- 4 leading zeros
- 5 leading zeros

For each difficulty level:

- Mine 30 blocks.
- Record:
 - Average mining time
 - Valid hash
 - Number of hash calculations (hash attempts)
- Calculate and compare the following performance metrics for each difficulty level:
 - Average mining time
 - Minimum mining time
 - Maximum mining time
 - Standard deviation of mining time
 - Mining throughput (hashes per second)

Your answer must include:

- Screenshots of your complete Python source code. **[6 marks]**
- Screenshots showing the output of your code. **[2 marks]**
- Present your results using a table that summaries all calculated performance metrics. **[2 marks]**
- A concise discussion interpreting your experimental results. **[2 marks]**

Answer the following questions based on your implementation, experimental results, and understanding of the underlying concepts. Support your answers with evidence where appropriate.

Reflection Questions:

1. Describe the relationship between mining difficulty and computational effort. Support your answer using the experimental results obtained in this assignment. **[3 marks]**
2. Compare the growth in mining time as the difficulty increases from 2 to 5 leading zeros. Is the increase approximately linear or exponential? Justify your answer using your experimental data. **[5 marks]**

3. Discuss one major limitation of the Proof-of-Work consensus mechanism and explain how an alternative consensus algorithm (e.g., Proof of Stake) attempts to address this limitation. **[3 marks]**

Exercise 3 **[12 marks]**: Elliptic Curve Cryptography

Generate an Elliptic Curve Digital Signature Algorithm (ECDSA) public/private key pair and save both keys in hexadecimal (or PEM) format. Copy and paste the generated private key (*private_ks*) and public key (*public_ks*) into a text editor for analysis. Perform the following tasks:

- Compare the size (in bytes and hexadecimal characters) of the private key and the public key.
- Research and implement a method to compress the public key. Display both the original and compressed public key and calculate the percentage reduction in storage size.
- Verify that the compressed public key can be correctly decompressed (or otherwise used) to validate a digital signature.

Your answer must include:

- Screenshots of your complete Python source code. **[5 marks]**
- Screenshots showing the output of your code. **[2 marks]**

Reflection Questions:

Answer the following questions based on your implementation, experimental results, and understanding of the underlying concepts. Support your answers with evidence where appropriate.

1. Why is it possible to compress an elliptic curve public key but not the corresponding private key? **[2 marks]**
2. If every public key stored on a blockchain were compressed, how would this affect blockchain storage requirements, network bandwidth, and transaction efficiency? **[3 marks]**

Part B

Exercise 4 **[20 marks]**: Blockchain Scalability, the Trilemma & Layer-2 Solutions

Bitcoin has a block size of 1 MB, an average transaction size of 580 bytes, and an average

block time of 576 seconds. Perform the following tasks:

1. Calculate the theoretical transaction throughput (TPS) of Bitcoin.
2. Calculate the TPS if the block size is increased to 2 MB while keeping the average block time unchanged.
3. Compare the throughput with one traditional payment system of your choice (e.g., Visa, PayPal, EFTPOS).
4. Select two blockchain scalability solutions from: SegWit, Lightning Network, Sharding, Sidechains.
5. Explain how each selected solution improves scalability.

Your answer must include:

- All calculations and workings. **[4 marks]**
- A comparison table summarising the scalability characteristics of Bitcoin and the selected payment system. **[4 marks]**
- A clear explanation of the two selected scalability solutions. **[4 marks]**

Reflection Questions

1. Discuss one advantage and one disadvantage of increasing the Bitcoin block size from 1 MB to 2 MB. **[3 marks]**
2. Using Bitcoin as an example, identify which assertions of the Blockchain Trilemma appear to be prioritized. Support your answer using characteristics of Bitcoin, and discuss whether these design choices are appropriate for Bitcoin's primary use case. **[5 marks]**

Exercise 5 [10 marks]: MetaMask, Wallets & Tokens

Using MetaMask and a public Ethereum test network (e.g., Sepolia):

- Create a wallet.
- Obtain test tokens.
- Complete at least one blockchain transaction.

Your answer must include:

- Wallet address used. **[2 marks]**
- Transaction hash. **[2 marks]**
- Screenshot showing the successful transaction. **[2 marks]**

Reflection Questions

1. Explain the role of the private key, public key, wallet address, and transaction hash in the transaction process. **[2 marks]**
2. Briefly describe one challenge you encountered while creating the wallet or completing the transaction. Explain how you resolved it, and what you learned from the experience. **[2 marks]**

Note: Use test tokens only. Do not use real funds.

Exercise 6 [10 marks]: Solidity Smart Contracts & Ethereum Development

Modify the Storage contract from the tutorial to store one additional variable relevant to a simple real-world application of your choice.

Examples: Student ID, Product ID, Asset Owner, Device Identifier, Course Code.

Perform the following tasks:

- Modify the smart contract.
- Compile and deploy the contract using Remix.
- Store and retrieve data using the new variable.

Your answer must include:

- Solidity source code. **[3 marks]**
- Contract address. **[1 mark]**
- Screenshot showing successful deployment. **[2 marks]**
- Evidence that the new variable can be successfully stored and retrieved. **[2 marks]**

Reflection Questions

1. Explain why you selected the additional variable and give one example of how it could be used in a real-world blockchain application. **[1 mark]**
2. Discuss one difference between a wallet address and a smart contract address. **[1 mark]**